LAPORAN TUGAS
ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Alfauzi Syahputra 2509106006

Kelas A ‘2025

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

BAB I

1.1 Soal ARRAY 1 Dimensi

1. Pseudocode

```
Judul :
Program bilangan prima, bilangan Fibonacci, dan bilangan Fibonacci prima

Kamus :
angka : array[1..100] of integer
prima : array[ ] of integer
fib : array[ ] of integer
i, j, k, n, pembagi : integer

Algoritma :
start
   print "Bilangan prima:"
   k = 0
   for i from 0 to 100 do
      pembagi = 0
      for j from 1 to angka[i] do
         if angka[i] % j == 0 then
            pembagi = pembagi + 1
      end for
      if pembagi == 2 then
         simpan angka[i] ke array prima[k]
         print angka[i]
         k = k + 1
   end for

   print "Bilangan Fibonacci:"
   fib[0] = 0
   fib[1] = 1
   print fib[0], fib[1]
   n = 2
   while fib[n-1] + fib[n-2] ≤ 100 do
      next = fib[n-1] + fib[n-2]
      simpan next ke array fib[n]
      print fib[n]
      n = n + 1
   end while

   print "Bilangan Fibonacci prima:"
   for i from 0 to n-1 do
      for j from 0 to k-1 do
         if fib[i] == prima[j] then
            print fib[i]
      end for
   end for
end
```

Gambar 1.1.1 Pseudocode

2. Algoritma

1) Inisialisasi Data Membuat sebuah array 1 dimensi bernama angka yang berisi deret bilangan bulat dari 0 sampai 100 sebagai sumber data utama.
2) Pencarian Bilangan Prima:
    a) Melakukan perulangan untuk memeriksa setiap elemen dalam array angka.
    b) Untuk setiap angka, hitung jumlah pembaginya. Jika pembagi hanya berjumlah 2 (angka 1 dan dirinya sendiri), maka angka tersebut adalah bilangan prima.
    c) Simpan angka prima yang ditemukan ke dalam array 1 dimensi baru bernama prima dan tampilkan ke layar.
3) Pembangkitan Bilangan Fibonacci:
    a) Menyiapkan array 1 dimensi fib dan menentukan nilai awal fib[0] = 0 dan fib[1] = 1.
    b) Menggunakan perulangan untuk menghitung angka berikutnya dengan menjumlahkan dua angka sebelumnya (fib[n-1] + fib[n-2]).
    c) Selama hasilnya tidak melebihi 100, simpan angka tersebut ke dalam array fib dan tampilkan ke layar.
4) Penyaringan Fibonacci Prima:
    a) Melakukan perulangan bersarang (nested loop) untuk membandingkan isi array fib dengan isi array prima.
    b) Jika ditemukan nilai yang sama (fib[i] == prima[j]), artinya angka tersebut adalah bilangan Fibonacci yang juga merupakan bilangan prima.
    c) Tampilkan hasil irisan tersebut ke layar.

3. Sourcecode

```
#include <iostream>
using namespace std;

int angka[101] = {
    0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
    11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
    21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
    31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
    41, 42, 43, 44, 45, 46, 47, 48, 49, 50,
    51, 52, 53, 54, 55, 56, 57, 58, 59, 60,
    61, 62, 63, 64, 65, 66, 67, 68, 69, 70,
    71, 72, 73, 74, 75, 76, 77, 78, 79, 80,
    81, 82, 83, 84, 85, 86, 87, 88, 89, 90,
    91, 92, 93, 94, 95, 96, 97, 98, 99, 100
};
```

Gambar 1.1.3.1 Source Code array 1 dimensi

Disini Program membuat Array 1 dimensi bertipe integer dengan index 101 yang berisi angka 1 sampai 100.

```
int main() {
    cout << "Bilangan Prima :" << endl;
    int prima[25];
    int k = 0;

    for (int i = 0; i < 101; i++) {
        int pembagi = 0;
        for (int j = 1; j <= angka[i]; j++) {
            if (angka[i] % j == 0)
                pembagi++;
        }

        if (pembagi == 2) {
            prima[k] = angka[i];
            cout << angka[i] << " ";
            k++;
        }
    }
```

Gambar 1.1.3.2 Source Code Bilangan Prima

Bagian source code ini menentukan apakah nilai variabel angka adalah bilangan prima atau bukan dengan cara membaginya dan jika pembagi == 2 nilai angka masuk ke array prima[k] yang dimana array 1 dimensi dan menampilkan nilai dari angka

```
    cout << "\n\nBilangan Fibonacci :" << endl;
    int fib[15];
    fib[0] = 0;
    fib[1] = 1;
    cout << fib[0] << " " << fib[1] << " ";

    int n = 2;
    while (true) {
        int next = fib[n - 1] + fib[n - 2];
        if (next > 100) break;

        fib[n] = next;
        cout << fib[n] << " ";
        n++;
    }
```

Gambar 1.1.3.3 Source Code Bilangan Fibonacci

Program untuk bilangan fibonacci dan nilai nya di simpan di array fib[n] setelah itu program menampilkan angka fibonacci.

```
    cout << "\n\nBilangan Fibonacci prima :" << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < k; j++) {
            if (fib[i] == prima[j]) {
                cout << fib[i] << " ";
                break;
            }
        }
    }

    cout << endl;
    return 0;
}
```

Gambar 1.1.3.4 Source Code Fibonacci Prima

Disini Program membandingkan nilai array fib dan nilai array prima dengan cara fib[i] == prima[j] terakhir program akan menampilkan angka fibonacci prima.

4. Output

```
Bilangan Prima :
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Bilangan Fibonacci :
0 1 1 2 3 5 8 13 21 34 55 89

Bilangan Fibonacci prima :
2 3 5 13 89
```

Gambar 1.1.4 Output

Deskripsi Tugas :

Buatlah Buatlah Program, Algoritma dan pseudocode yang menggunakan array 1 dimensi (boleh ditambahkan percabangan / perulangan)

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

1.2 Soal Nomor 1 : Min, Max dan Average

1. Pseudocode

```
Judul :
Program Nilai Min, Max, dan Average untuk Matrix

Kamus :
matrix : array [{6, 7, 8},{1, 5, 3}] of integer
i, j, rows, cols, total : integer

Algoritma :
start
   for i from 0 to rows-1 do
      for j from 0 to cols-1 do
         if matrix[i][j] < min_value then
            min_value = matrix[i][j]
         end if
         if matrix[i][j] > max_value then
            max_value = matrix[i][j]
         end if
         total = total + matrix[i][j]
      end for
   end for

   average = total / (rows * cols)

   print "Matriks:"
   for i from 0 to rows-1 do
      for j from 0 to cols-1 do
         print matrix[i][j] + " "
      end for
      print new line
   end for

   print "Nilai minimum  : " + min_value
   print "Nilai maksimum : " + max_value
   print "Rata-rata      : " + average
end
```

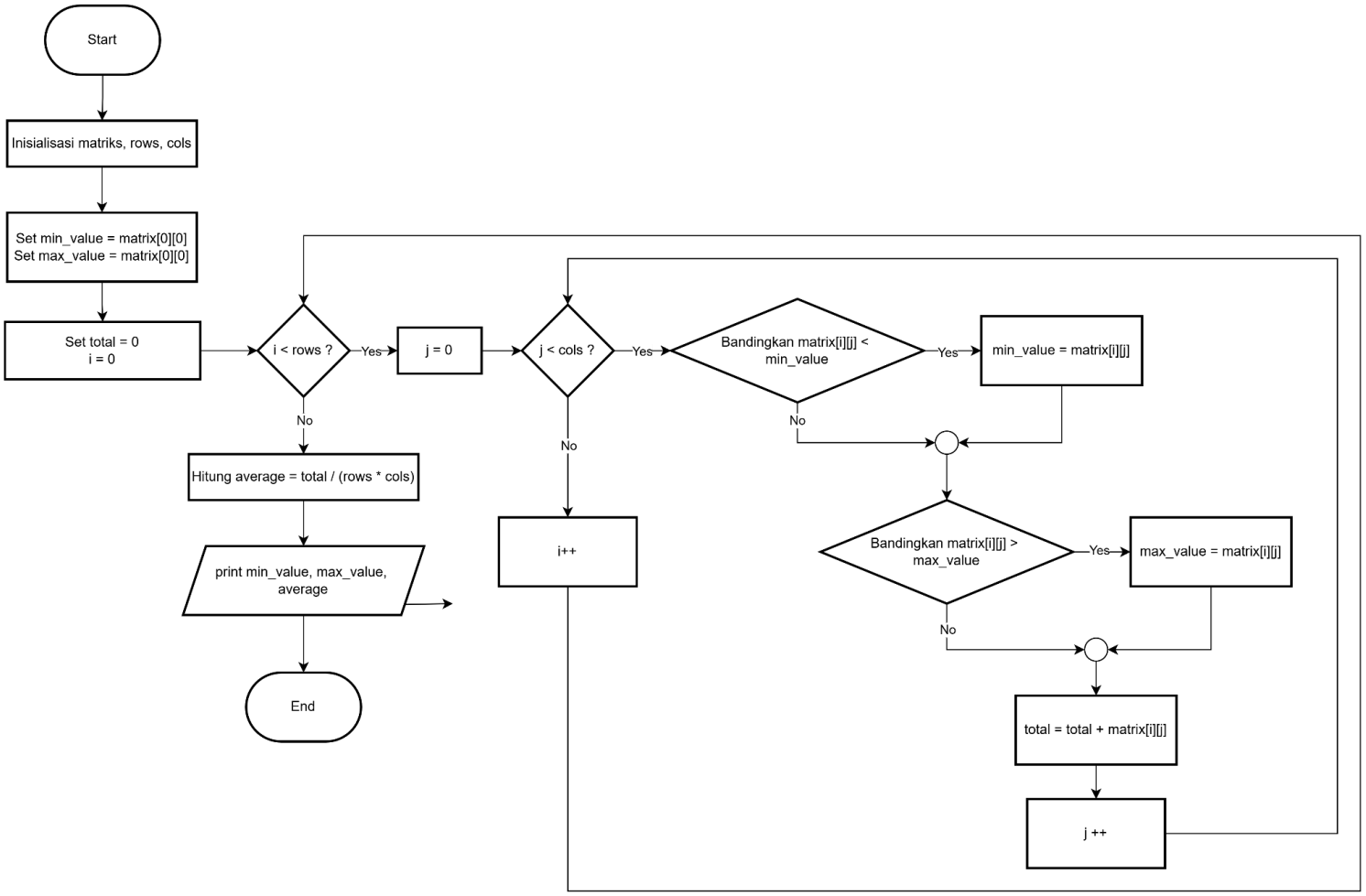
Gambar 1.2.1 Pseudocode

2. Algoritma

1) Inisialisasi Matriks dan Variabel
   a) Mendefinisikan sebuah matriks dua dimensi berukuran 2 x 3 dengan elemen-elemen statis yaitu {6, 7, 8} pada baris pertama dan {1, 5, 3} pada baris kedua.
   b) Menentukan jumlah baris (rows = 2) dan jumlah kolom (cols = 3).
   c) Menentukan nilai awal variabel min_value dan max_value dengan mengambil elemen pertama matriks, yaitu matrix[0][0].
   d) Menyiapkan variabel total dengan nilai awal 0 untuk menampung jumlah keseluruhan elemen.

2) Proses Pemindaian (*Scanning*) Matriks
   a) Melakukan perulangan bersarang (*nested loop*) untuk mengakses setiap sel di dalam matriks.
   b) Mencari Nilai Minimum Di setiap sel, bandingkan nilai elemen saat ini dengan min_value. Jika elemen tersebut lebih kecil, maka perbarui nilai min_value.
   c) Mencari Nilai Maksimum Bandingkan nilai elemen saat ini dengan max_value. Jika elemen tersebut lebih besar, maka perbarui nilai max_value.
   d) Akumulasi Total Tambahkan nilai elemen yang sedang diakses ke dalam variabel total.
3) Setelah seluruh elemen matriks selesai diproses, hitung nilai rata-rata (average) dengan membagi variabel total dengan hasil kali jumlah baris dan kolom (2 x 3 = 6).
4) Menampilkan kembali bentuk matriks ke layar agar visualisasinya jelas dan Menampilkan hasil akhir yang meliputi nilai terkecil (minimum), nilai terbesar (maksimum), dan nilai rata-rata yang telah dihitung.

3. Flowchart



Gambar 1.2.3 Flowchart

4. Sourcecode

```
#include <iostream>
using namespace std;

int matrix[2][3] = {
        {6, 7, 8},
        {1, 5, 3}
};

const int rows = 2;
const int cols = 3;

int min_value = matrix[0][0];
int max_value = matrix[0][0];
int total = 0;
```

Gambar 1.2.4.1 Source Code Matrix

Pertama Program Membuat Matrik 2 x 3 yang berisi nilai tipe integer setelah itu membuat variabel rows dan cols yang dimana bertipe integer dan tetap, total = 0 bertipe integer, variabel min_value dan max_value adalah nilai matrix di index[0][0] yang nanti akan digunakan di kode blok berikutnya.

```
int main() {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] < min_value) {
                min_value = matrix[i][j];
            }
            if (matrix[i][j] > max_value) {
                max_value = matrix[i][j];
            }
            total += matrix[i][j];
        }
    }
```

Gambar 1.2.4.2 Source Code Nilai Min dan Max

Program ini untuk menentukan nilai min dan max dari matrix dengan cara membandingkan nilai matriksnya untuk min menggunakan matrix[i][j] < min_value dan untuk max menggunakan matrix[i][j] > max_value dan ada variabel total untuk digunakan menghitung average.

```
    float average = static_cast<float>(total) / (rows * cols);

    cout << "Matriks:" << endl;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    cout << "\nNilai minimum  : " << min_value << endl;
    cout << "Nilai maksimum : " << max_value << endl;
    cout << "Rata-rata      : " << average << endl;

    return 0;
}
```

Gambar 1.2.4.3 Source Code Average dan output

Bagian ini untuk menghitung nilai rata-rata matrik yaitu average = (total) / (rows * cols) setelah itu program menampilkan Output matrik, min, max, dan average.

5. Output

```
Matriks:
6       7       8
1       5       3

Nilai minimum  : 1
Nilai maksimum : 8
Rata-rata      : 5
```

Gambar 1.2.5 Output

Deskripsi Tugas :

Buatlah pseudocode, algoritma, flowchart dan program untuk menampilkan nilai min, max dan rata-rata dalam suatu matriks.

Contoh: Bila ada matriks berikut:

6 7 8

1 5 3

Maka nilai terkecil adalah 1, nilai maksimum adalah 8, dan rata-rata adalah 5.

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

1.3 Soal Nomor 2 : Invers Matriks

1. Pseudocode

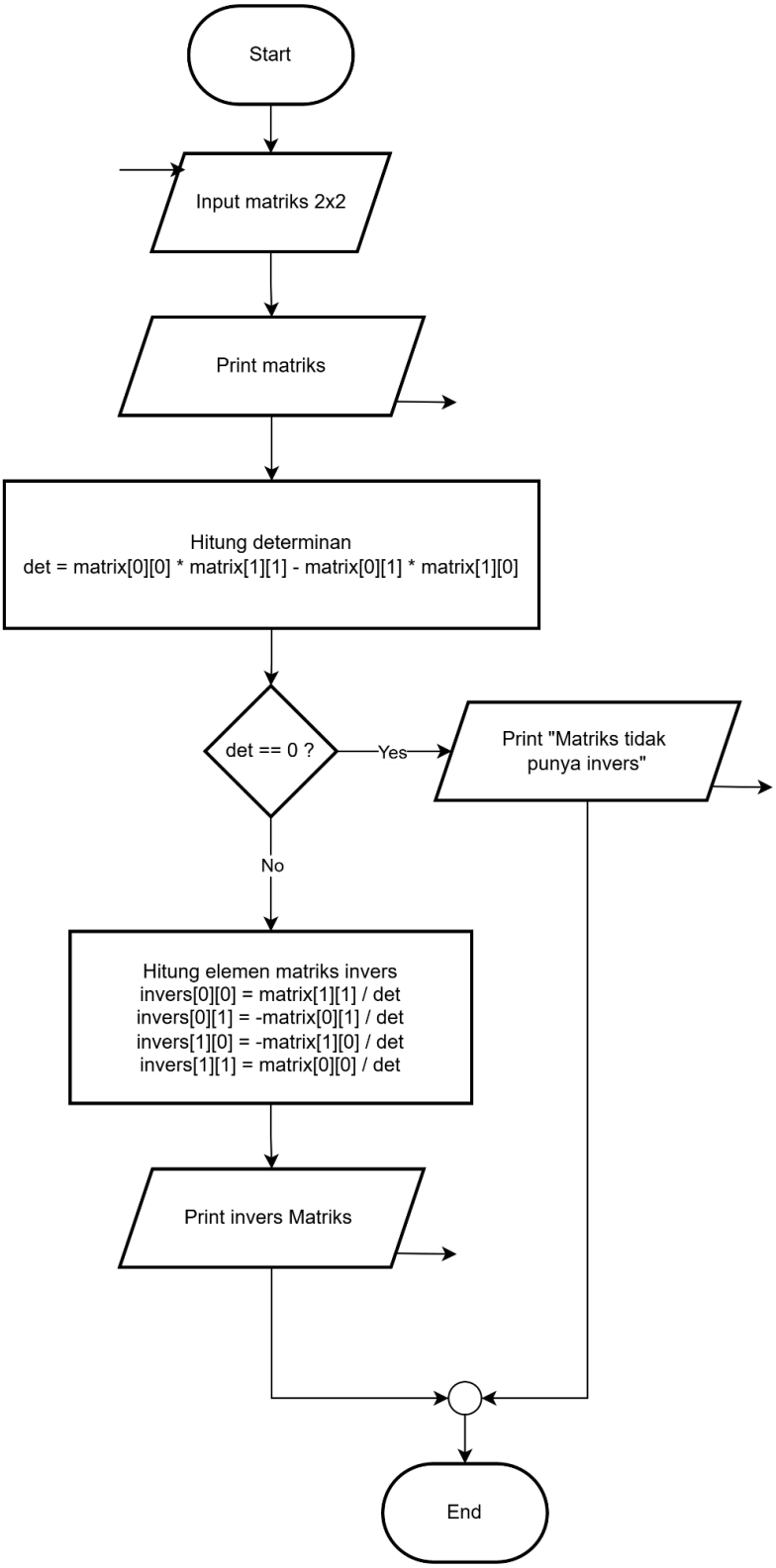
```
Judul :
Program Menghitung Invers Matriks Berordo 2x2

Kamus :
matrix : array [2][2] of real
invers : array [2][2] of real
det : real
i, j : integer

Algoritma :
start
   print "Masukkan elemen matriks 2x2:"
   for i from 0 to 1 do
      for j from 0 to 1 do
         print "Elemen [", i, "][", j, "]: "
         input matrix[i][j]
      end for
   end for

   print "Matriks:"
   for i from 0 to 1 do
      for j from 0 to 1 do
         print matrix[i][j]
      end for
   end for

   det = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]

   if det == 0 then
      print "Matriks tidak memiliki invers (determinan = 0)."
   else
      invers[0][0] = matrix[1][1] / det
      invers[0][1] = -matrix[0][1] / det
      invers[1][0] = -matrix[1][0] / det
      invers[1][1] = matrix[0][0] / det

      print "Matriks invers:"
      for i from 0 to 1 do
         for j from 0 to 1 do
            print invers[i][j]
         end for
      end for
   end if
end
```

Gambar 1.3.1 Pseudocode

2. Algoritma

1) Mendeklarasikan matriks dua dimensi matrix[2][2] untuk menampung input pengguna dan variabel det untuk menyimpan nilai determinan, keduanya menggunakan tipe data double agar dapat mengolah bilangan pecahan.
2) Input Data Elemen meminta pengguna memasukkan nilai untuk empat elemen matriks (a, b, c, d) melalui perulangan.
3) Menampilkan kembali matriks yang diinput ke layar dalam format baris dan kolom.
4) Menghitung nilai determinan dengan rumus: det = (a \times d) - (b \times c) atau dalam indeks kode (matrix[0][0] * matrix[1][1]) - (matrix[0][1] * matrix[1][0]).
5) Pengecekan Kondisi (Percabangan):
    a) Jika determinan sama dengan 0 rogram akan menampilkan pesan bahwa matriks tidak memiliki invers (matriks singular).
    b) Jika determinan tidak sama dengan 0 Program melanjutkan ke proses perhitungan invers.
6) Perhitungan Invers Matriks
    a) Menukar posisi elemen a dengan d.
    b) Mengalikan elemen b dan c dengan -1.
    c) Membagi seluruh elemen yang telah diubah tersebut dengan nilai determinan.
7) Menampilkan elemen-elemen matriks invers yang telah berhasil dihitung ke layar secara terstruktur.

3. Flowchart



Gambar 1.3.3 Flowchart

4. Sourcecode

```
#include <iostream>
using namespace std;

int main() {
    double matrix[2][2];
    double det;

    cout << "Masukkan elemen matriks 2x2:\n";
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << "Elemen [" << i << "][" << j << "]: ";
            cin >> matrix[i][j];
        }
    }

    cout << "\nMatriks:\n";
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }
```

Gambar 1.3.4.1 Source Code input nilai matrix

Di bagian ini, program meminta pengguna untuk memasukkan elemen matriks 2x2 satu per satu menggunakan perulangan for. Setelah data diinput, program akan menampilkan kembali matriks tersebut ke layar agar pengguna dapat melihat susunan angka yang telah dimasukkan.

```
    det = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0];

    if (det == 0) {
        cout << "\nMatriks tidak memiliki invers (determinan = 0).\n";
    } else {
        double invers[2][2];

        invers[0][0] = matrix[1][1] / det;
        invers[0][1] = -matrix[0][1] / det;
        invers[1][0] = -matrix[1][0] / det;
        invers[1][1] = matrix[0][0] / det;
```

Gambar 1.3.4.2 Source Code Determinan dan invers

Program menghitung nilai determinan menggunakan rumus $ad - bc$. Jika determinan tidak nol, program akan menghitung elemen matriks invers dengan cara menukar posisi elemen diagonal utama, mengubah tanda elemen diagonal samping, dan membagi semuanya dengan nilai determinan.

```
        cout << "\nMatriks invers:\n";
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                cout << invers[i][j] << "\t";
            }
            cout << endl;
        }
    }

    return 0;
}
```

Gambar 1.3.4.3 Source Code Output Invers Matrix

Bagian ini berfungsi untuk menampilkan hasil perhitungan matriks invers ke layar. Program menggunakan perulangan for untuk mencetak elemen-elemen matriks invers[i][j] secara rapi dengan bantuan tabulasi (\t).

5. Output

```
Masukkan elemen matriks 2x2:
Elemen [0][0]: 1
Elemen [0][1]: 2
Elemen [1][0]: 1
Elemen [1][1]: 3

Matriks:
1       2
1       3

Matriks invers:
3       -2
-1      1
```

Gambar 1.3.5 Output

Deskripsi Tugas :

Buatlah pseudocode, algoritma, flowchart dan program untuk menghitung invers matriks berordo 2x2.

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

1.4 Soal Nomor 3 : Array Multidimensi

1. Pseudocode

```
Judul :
Program Array Multidimensi Dan Nilai Average

Kamus :
rows, cols : integer
matrix : array [rows][cols] of real
total, average : real
i, j : integer

Algoritma :
start
   input rows, cols
   if rows <= 0 or cols <= 0 then
      print "Error: Dimensi array harus bernilai positif!"
      exit
   end if

   total = 0

   for i from 0 to rows-1 do
      for j from 0 to cols-1 do
         print "Elemen [", i, "][", j, "]: "
         input matrix[i][j]
         total = total + matrix[i][j]
      end for
   end for

   print "Matriks:"
   for i from 0 to rows-1 do
      for j from 0 to cols-1 do
         print matrix[i][j], "\t"
      end for
      print new line
   end for

   average = total / (rows * cols)

   print "Total elemen   : ", rows * cols
   print "Jumlah nilai   : ", total
   print "Rata-rata      : ", average
end
```

Gambar 1.4.1 Pseudocode

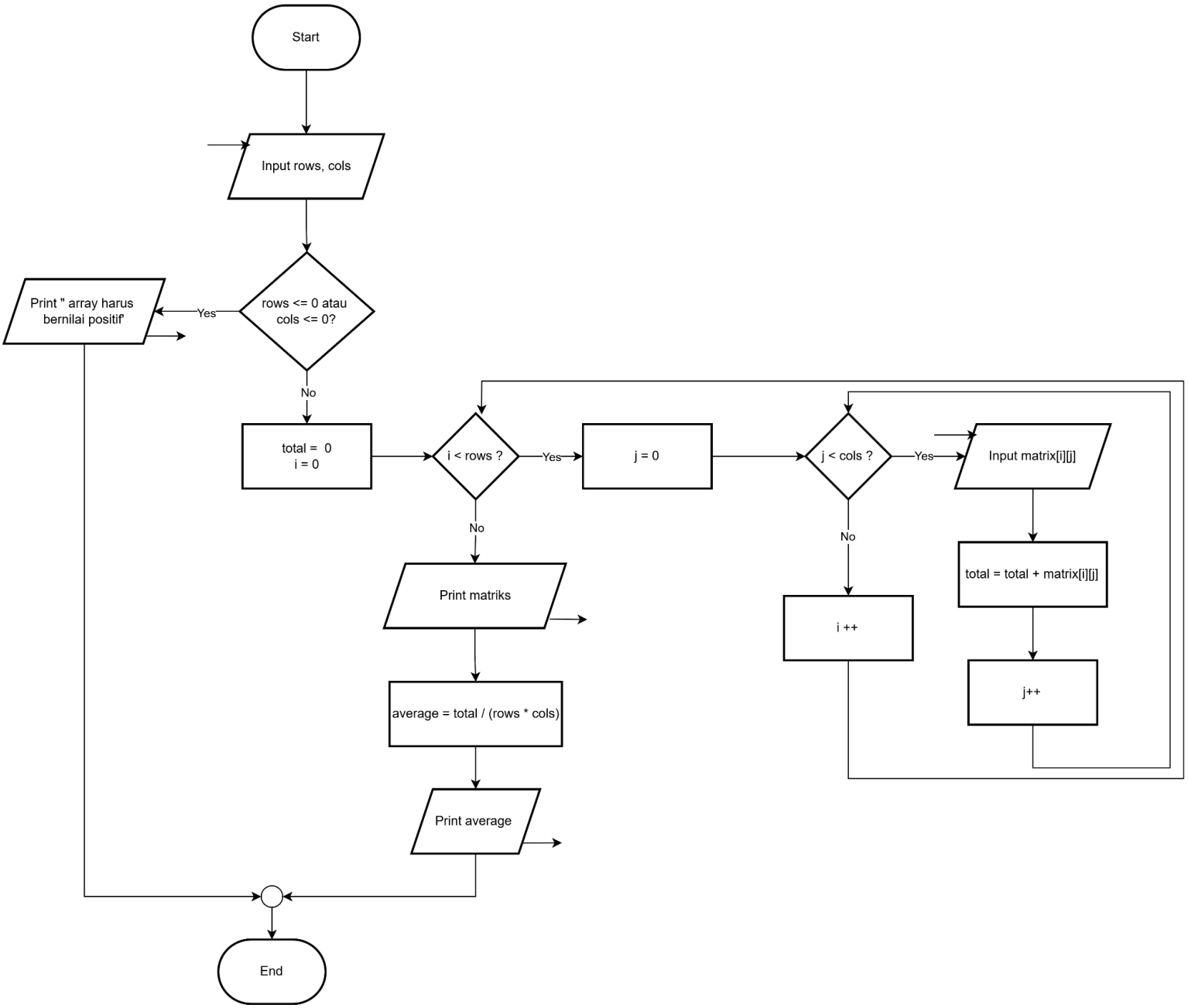
2. Algoritma

1) Memulai program dengan meminta input jumlah baris dan kolom dari pengguna.
2) Memeriksa apakah input dimensi bernilai positif. Jika tidak, program memberikan peringatan error.
3) Inisialisasi & Pengisian
    a) Membuat matriks dua dimensi dengan tipe data double.

b) Melakukan iterasi untuk mengisi setiap elemen matriks.
c) Menghitung akumulasi jumlah seluruh nilai elemen ke dalam variabel total.

4) Menampilkan isi matriks ke layar agar mempermudah verifikasi data.
5) Perhitungan Statistik:
   a) Menghitung total elemen yang ada (baris x kolom).
   b) Menghitung rata-rata elemen (total / jumlah elemen).
6) Menampilkan total elemen, jumlah nilai, dan hasil rata-rata.

3. Flowchart



Gambar 1.4.3 Flowchart

4. Sourcecode

```
#include <iostream>
using namespace std;

int main() {
    int rows, cols;

    cout << "Masukkan jumlah baris: ";
    cin >> rows;
    cout << "Masukkan jumlah kolom: ";
    cin >> cols;

    if (rows <= 0 || cols <= 0) {
        cerr << "\nError: Dimensi array harus bernilai positif!" << endl;
        return 1;
    }
```

Gambar 1.4.4.1 Source Code Input rows dan cols

Bagian ini berfungsi untuk menentukan dimensi matriks secara dinamis. Program meminta pengguna memasukkan jumlah baris dan kolom, lalu melakukan validasi. Jika angka yang dimasukkan kurang dari atau sama dengan 0, program akan menampilkan pesan kesalahan dan berhenti.

```
    double matrix[rows][cols];
    double total = 0.0;

    cout << "\nMasukkan elemen matriks:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << "Elemen [" << i << "][" << j << "]: ";
            cin >> matrix[i][j];
            total += matrix[i][j];
        }
    }
```

Gambar 1.4.4.2 Source Code input elemen matrix

Program mendeklarasikan matriks dengan ukuran yang telah ditentukan sebelumnya. Menggunakan perulangan bersarang (nested loop), program meminta pengguna mengisi setiap sel matriks. Bersamaan dengan proses input, nilai setiap elemen langsung dijumlahkan ke dalam variabel total.

```
    cout << "\nMatriks:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }

    double average = total / (rows * cols);

    cout << "\nTotal elemen   : " << rows * cols << endl;
    cout << "Jumlah nilai   : " << total << endl;
    cout << "Rata-rata      : " << average << endl;

    return 0;
}
```

Gambar 1.4.4.3 Source Code output dan average

Bagian akhir ini digunakan untuk menampilkan kembali matriks dalam format yang rapi menggunakan tabulasi (\t). Program kemudian menghitung nilai rata-rata dengan membagi total dengan jumlah seluruh elemen (rows * cols) dan menampilkan hasilnya ke layar.

5. Output

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 2

Masukkan elemen matriks:
Elemen [0][0]: 1
Elemen [0][1]: 3
Elemen [1][0]: 2
Elemen [1][1]: 4
Elemen [2][0]: 8
Elemen [2][1]: 6

Matriks:
1       3
2       4
8       6

Total elemen   : 6
Jumlah nilai   : 24
Rata-rata      : 4
```

Gambar 1.4.5 Output

Deskripsi Tugas :

Buatlah pseudocode, algoritma, flowchart dan program untuk menampilkan array multidimensi dan menampilkan nilai rata-rata elemen yang ada didalamnya.

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

1.5 Soal Nomor 4 : Pointer

1. Sourcecode

```
#include <iostream>
using namespace std;

struct Player {
    string name;
    int score;
    int hp;
};
```

Gambar 1.5.1.1 Source Code struct Player

Di bagian ini, program mendefinisikan sebuah Struktur (struct) bernama Player. Struct adalah tipe data bentukan yang mengelompokkan beberapa variabel dengan tipe berbeda menjadi satu kesatuan. Di sini, setiap "Player" akan memiliki tiga atribut: name (string), score (integer), dan hp (integer). Ini memudahkan pengelolaan data objek yang memiliki banyak informasi.

```
void displayPlayer(Player *p) {
    cout << "Nama: " << p->name << endl;
    cout << "Score: " << p->score << endl;
    cout << "HP: " << p->hp << endl;
    cout << "-----------------------\n";
}
```

Gambar 1.5.1.2 Source Code Prosedur displayplayer

Program membuat prosedur displayPlayer yang menggunakan parameter Pointer (Player *p). Penggunaan tanda bintang (*) menunjukkan bahwa prosedur ini tidak menerima salinan data objek, melainkan alamat memori dari objek tersebut. Di dalamnya, operator panah (->) digunakan untuk mengakses anggota struct melalui pointer. Teknik ini lebih efisien karena program tidak perlu menduplikasi data yang besar ke dalam fungsi.

```
int main() {
    Player player1 = {"Warrior", 100, 50};
    Player *ptr1 = &player1;

    Player player2 = {"Mage", 120, 40};
    Player *ptr2 = &player2;

    cout << "Data Player 1:\n";
    displayPlayer(ptr1);

    cout << "Data Player 2:\n";
    displayPlayer(ptr2);

    return 0;
}
```

Gambar 1.5.1.3 Source Code main()

Pada gambar 1.5.1.3 dimana program utama (main) menginisialisasi dua objek yaitu player1 dan player2. Kemudian, dibuat variabel pointer ptr1 dan ptr2 yang diisi dengan alamat memori objek menggunakan operator address-of (&). Saat memanggil displayPlayer(ptr1), program mengirimkan "peta lokasi" data di memori, sehingga fungsi tersebut dapat mencetak informasi pemain secara langsung dari sumber aslinya.

2. **Output**

```
Data Player 1:
Nama: Warrior
Score: 100
HP: 50
-----------------------
Data Player 2:
Nama: Mage
Score: 120
HP: 40
-----------------------
```

Gambar 1.5.2 Output

Deskripsi Tugas :
Buatlah program menggunakan pointer.

Nim dan Nama yang mengerjakan :
2509106006 - Muhammad Alfauzi Syahputra

1.6 Soal Nomor 5 : Matriks Segitiga Atas Dan Bawah

1. Sourcecode

```
int main() {
    int n = 5;
    int matrix[5][5];
```

Gambar 1.6.1.1 Source Code ukuran index matrix

Di sini program melakukan inisialisasi variabel n dengan nilai 5, yang menentukan bahwa matriks yang akan dibuat adalah matriks persegi berukuran 5×5. Program kemudian mendeklarasikan array dua dimensi matrix[5][5] untuk menampung angka-angka pola yang akan dihasilkan.

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i + j < n - 1) {
                matrix[i][j] = 9;
            } else if (i + j > n - 1) {
                matrix[i][j] = 1;
            } else {
                matrix[i][j] = 0;
            }
        }
    }
```

Gambar 1.6.1.2 Source Code Logika Segitiga

Ini adalah inti dari program. Program menggunakan perulangan bersarang (nested loop) untuk menelusuri setiap koordinat baris (i) dan kolom (j). Penentuan angka menggunakan logika Diagonal Sekunder ($i + j = n - 1$)

- Kondisi i + j < n - 1: Jika jumlah indeks baris dan kolom lebih kecil dari 4, maka posisi tersebut berada di atas diagonal sekunder, sehingga diisi dengan angka 9.
- Kondisi i + j > n - 1: Jika jumlahnya lebih besar dari 4, posisi berada di bawah diagonal sekunder, diisi dengan angka 1
- Kondisi else: Jika jumlahnya tepat sama dengan 4, maka posisi tersebut adalah garis diagonal itu sendiri, diisi dengan angka 0.

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Gambar 1.6.1.3 Source Code Output

Pada gambar 1.6.1.3 source code untuk mencetak isi matriks ke layar terminal. Perulangan baris mengontrol perpindahan baris baru (cout << endl), sementara perulangan kolom mencetak angka beserta spasi agar membentuk susunan persegi yang rapi sesuai dengan pola segitiga yang telah diproses.

2. **Output**

```
9 9 9 9 0
9 9 9 0 1
9 9 0 1 1
9 0 1 1 1
0 1 1 1 1
```

Gambar 1.6.2 Output

Deskripsi Tugas :

Buatlah program untuk menampilkan matriks segitiga atas dan bawah seperti ini:

9 9 9 9 0
9 9 9 0 1
9 9 0 1 1
9 0 1 1 1
0 1 1 1 1

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

BAB II

1. Pseudocode

```
Judul :
Program Prosedur Untuk Menentukan Sisi Sebuah Segitiga

Kamus :
sisi1, sisi2, sisi3 : real
a, b, c : real ← untuk mengurutkan sisi
sisi_terpanjang_kuadrat, jumlah_kuadrat_lainnya : real

Algoritma :
start
   if sisi1 >= sisi2 and sisi1 >= sisi3 then
      a = sisi2; b = sisi3; c = sisi1
   else if sisi2 >= sisi1 and sisi2 >= sisi3 then
      a = sisi1; b = sisi3; c = sisi2
   else
      a = sisi1; b = sisi2; c = sisi3

   if a + b <= c then
      print "Tidak bisa membentuk segitiga"
   else
      sisi_terpanjang_kuadrat = c * c
      jumlah_kuadrat_lainnya = a*a + b*b

      if sisi_terpanjang_kuadrat == jumlah_kuadrat_lainnya then
         print "Segitiga siku-siku"
      else if a == b or b == c or a == c then
         print "Segitiga sama kaki"
      else
         print "Segitiga sembarang"
      end if
   end if
end
```

Gambar 2.1 Pseudocode

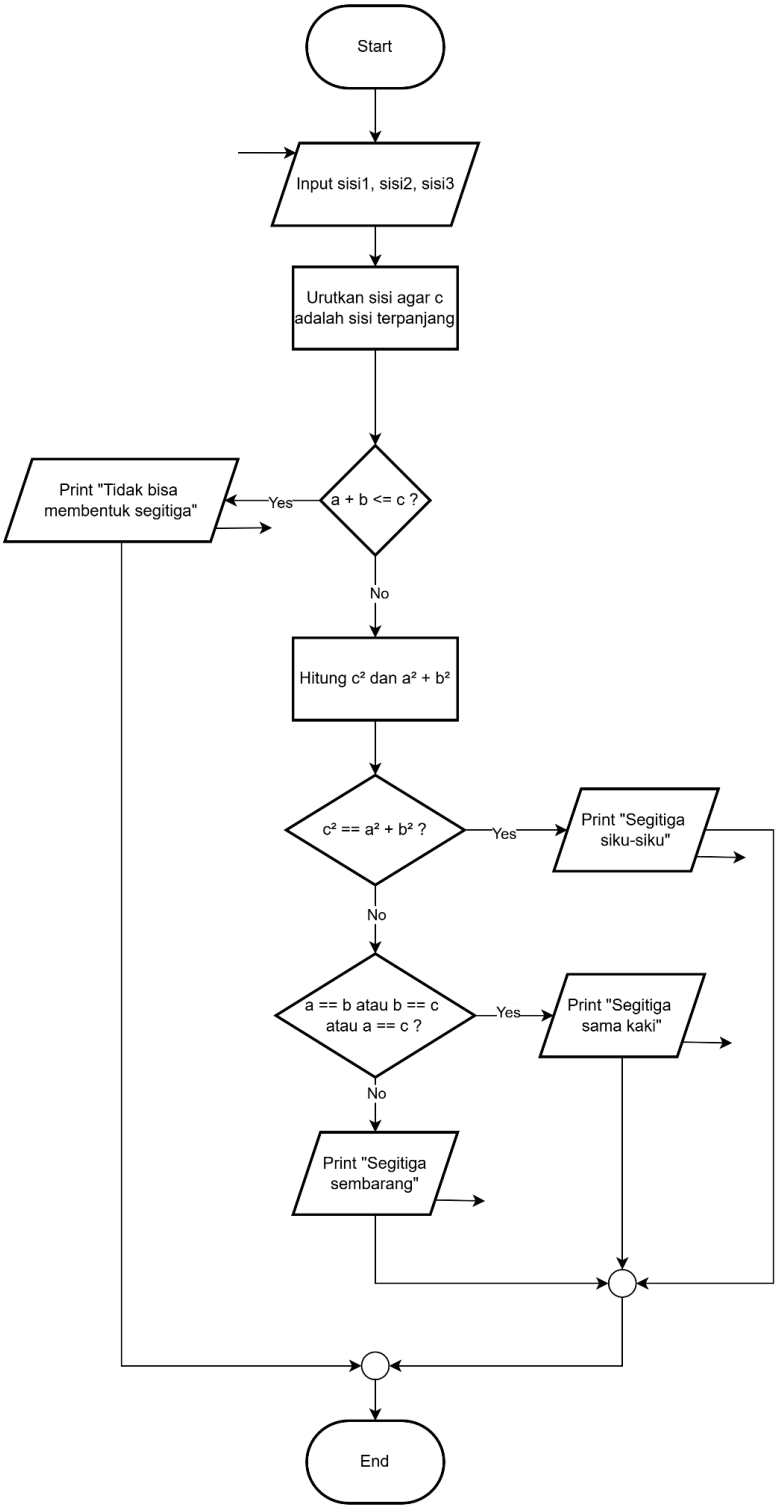
2. Algoritma

1) Program meminta tiga input nilai numerik yang merepresentasikan panjang sisi-sisi sebuah calon segitiga.
2) Identifikasi Sisi Terpanjang: Membandingkan ketiga sisi untuk menentukan mana yang paling panjang dan menempatkannya ke variabel khusus (c).
3) Memeriksa apakah sisi a + b > c Jika syarat ini tidak terpenuhi, tampilkan pesan bahwa sisi tersebut tidak dapat membentuk segitiga dan hentikan proses.
4) Identifikasi Karakteristik
   a) Hitung kuadrat dari masing-masing sisi. Jika kuadrat sisi miring sama dengan jumlah kuadrat sisi lainnya, maka termasuk Segitiga Siku-siku.
   b) Jika terdapat dua sisi yang nilainya identik, maka termasuk Segitiga Sama Kaki.

c) Jika semua sisi berbeda dan tidak memenuhi hukum Pythagoras, maka termasuk Segitiga Sembarang.

5) Menampilkan hasil klasifikasi jenis segitiga ke layar.

3. Flowchart



Gambar 2.3 Flowchart

4. Sourcecode

```
#include <iostream>
using namespace std;

void tentukanJenisSegitiga(double sisi1, double sisi2, double sisi3) {
    double a, b, c;

    if (sisi1 >= sisi2 && sisi1 >= sisi3) {
        a = sisi2; b = sisi3; c = sisi1;
    } else if (sisi2 >= sisi1 && sisi2 >= sisi3) {
        a = sisi1; b = sisi3; c = sisi2;
    } else {
        a = sisi1; b = sisi2; c = sisi3;
    }
```

Gambar 2.4.1 Source Code Prosedur tentukan jenis segitiga, logika untuk c sisi selalu terpanjang

Bagian ini merupakan blok awal dari prosedur tentukanJenisSegitiga. Program melakukan pengurutan (sorting) manual terhadap tiga inputan sisi. Logika ini sangat penting untuk memastikan variabel c selalu menyimpan nilai sisi terpanjang, sementara a dan b menyimpan sisi lainnya. Hal ini dilakukan agar pengujian syarat terbentuknya segitiga dan rumus Pythagoras pada tahap berikutnya menjadi akurat dan konsisten tanpa mempedulikan urutan input dari pengguna.

```
    if (a + b <= c) {
        cout << "Tidak bisa membentuk segitiga." << endl;
    } else {
        double c_kuadrat = c * c;
        double ab_kuadrat = a * a + b * b;

        if (c_kuadrat == ab_kuadrat) {
            cout << "Segitiga siku-siku." << endl;
        } else if (a == b || b == c || a == c) {
            cout << "Segitiga sama kaki." << endl;
        } else {
            cout << "Segitiga sembarang." << endl;
        }
    }
}
```

Gambar 2.4.2 Source Code Prosedur tentukan jenis segitiga, logika jenis segitiga

Setelah sisi terpanjang ditentukan, program melakukan validasi menggunakan Ketaksamaan Segitiga jika jumlah dua sisi terpendek (a + b) tidak lebih besar dari sisi terpanjang (c), maka program menyatakan tidak bisa membentuk segitiga. Jika valid, program masuk ke logika percabangan untuk menentukan jenisnya:

1) Siku-siku Menggunakan hukum Pythagoras ($c^2 = a^2 + b^2$).
2) Sama Kaki Mengecek apakah ada dua sisi yang memiliki panjang yang sama.
3) Sembarang Jika tidak memenuhi kriteria di atas, maka dikategorikan sebagai segitiga sembarang.

```
int main() {
    double sisi1, sisi2, sisi3;

    cout << "Masukkan sisi 1: ";
    cin >> sisi1;
    cout << "Masukkan sisi 2: ";
    cin >> sisi2;
    cout << "Masukkan sisi 3: ";
    cin >> sisi3;

    tentukanJenisSegitiga(sisi1, sisi2, sisi3);

    return 0;
}
```

Gambar 2.4.3 Source Code main(), input sisi 1,2,3 dan Output

Di dalam fungsi utama (main), program mendeklarasikan variabel bertipe double agar dapat menerima input angka desimal. Program meminta pengguna memasukkan nilai ketiga sisi, kemudian memanggil prosedur tentukanJenisSegitiga dengan mengirimkan ketiga nilai tersebut sebagai argumen. Hasil akhir berupa informasi jenis segitiga kemudian ditampilkan langsung melalui prosedur tersebut.

5. Output

```
Masukkan sisi 1: 3
Masukkan sisi 2: 4
Masukkan sisi 3: 5
Segitiga siku-siku.
```

Gambar 2.5 Output

Deskripsi Tugas :

1) Tulislah sebuah prosedur untuk menentukan sisi sebuah segitiga.
2) Terdapat 3 inputan yaitu sisi 1, sisi 2, sisi 3.
3) Tentukan dari ketiga sisi tersebut dapat terbentuk segitiga atau tidak.
4) Jika tidak bisa, keluarkan informasi tidak bisa.
5) Jika bisa, tentukan apakah segitiga siku-siku, segitiga sembarang, atau segitiga sama kaki.
6) Buatlah pseudocode, flowchart dan program.

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

BAB III

1. Pseudocode

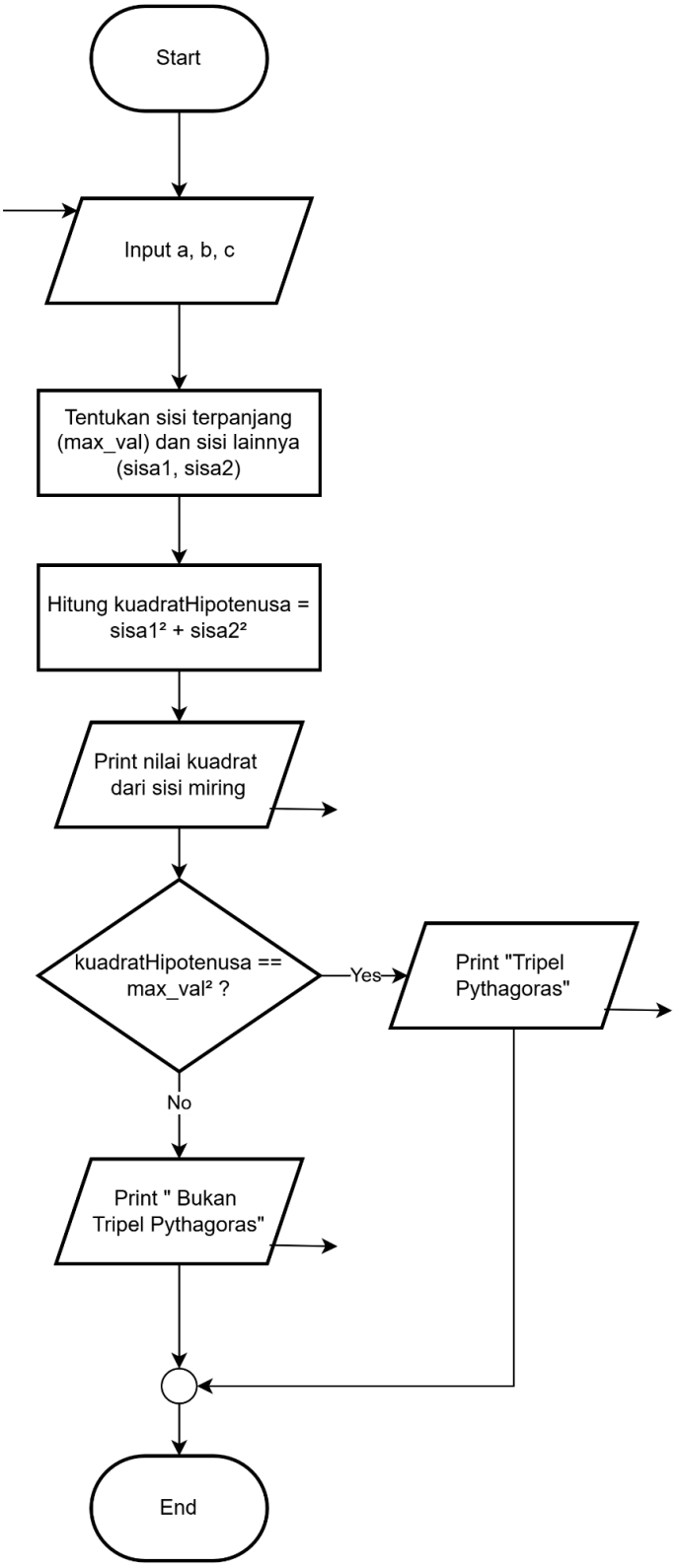
```
Judul :
Program Fungsi untuk Menentukan Nilai dan Mengecek Tripel Pythagoras

Kamus :
a, b, c : integer
max_val, sisi1, sisi2 : integer
kuadratHipotenusa : integer

start
   input a, b, c
   if a >= b and a >= c then
      max_val = a; sisa1 = b; sisa2 = c
   else if b >= a and b >= c then
      max_val = b; sisa1 = a; sisa2 = c
   else
      max_val = c; sisa1 = a; sisa2 = b

   kuadratHipotenusa = sisa1 * sisa1 + sisa2 * sisa2

   print "Nilai kuadrat dari sisi miring (hipotenusa) = " + kuadratHipotenusa

   if kuadratHipotenusa == (max_val * max_val) then
      print "Tripel Pythagoras"
   else
      print "Bukan Tripel Pythagoras"
   end if
end
```

Gambar 3.1 Pseudocode

2. Algoritma

1) Program menerima tiga input bilangan bulat dari pengguna sebagai calon sisi segitiga.
2) Mencari nilai terbesar dari ketiga bilangan tersebut untuk ditetapkan sebagai sisi miring (c).
3) Perhitungan Kuadrat:
   a) Menghitung kuadrat dari sisi terbesar (c^2).
   b) Menghitung jumlah kuadrat dari dua sisi lainnya ($a^2 + b^2$).
4) Verifikasi Tripel Pythagoras Membandingkan hasil kedua perhitungan tersebut.
   a) Jika $a^2 + b^2 = c^2$, maka statusnya adalah True (Tripel Pythagoras).
   b) Jika $a^2 + b^2 \neq c^2$, maka statusnya adalah False (Bukan Tripel Pythagoras).
5) Menampilkan hasil perhitungan kuadrat hipotenusa dan memberikan kesimpulan apakah input tersebut termasuk Tripel Pythagoras atau tidak.

3. Flowchart



Gambar 3.3 Flowchart

4. Sourcecode

```
#include <iostream>
using namespace std;

bool cekTripelPythagoras(int a, int b, int c) {
    int max_val, sisi1, sisi2;

    if (a >= b && a >= c) {
        max_val = a; sisi1 = b; sisi2 = c;
    } else if (b >= a && b >= c) {
        max_val = b; sisi1 = a; sisi2 = c;
    } else {
        max_val = c; sisi1 = a; sisi2 = b;
    }
```

Gambar 3.4.1 Source Code Fungsi cekTripelPythagoras, Logika c

Bagian ini mendefinisikan fungsi cekTripelPythagoras yang mengembalikan nilai boolean (true atau false). Di dalam fungsi ini, program melakukan penyaringan awal untuk menentukan angka terbesar di antara tiga inputan (a, b, c). Angka terbesar tersebut disimpan dalam variabel max_val yang diasumsikan sebagai sisi miring (hipotenusa), sedangkan dua angka lainnya disimpan dalam sisi1 dan sisi2. Hal ini penting agar rumus Pythagoras tetap valid meskipun pengguna memasukkan angka secara acak.

```
    int kuadratHipotenusa = sisi1 * sisi1 + sisi2 * sisi2;

    cout << "Nilai kuadrat dari sisi miring (hipotenusa) = " << kuadratHipotenusa << endl;

    return kuadratHipotenusa == (max_val * max_val);
}
```

Gambar 3.4.2 Source Code Fungsi cekTripelPythagoras, kuadratHipotenusa

Setelah sisi-sisi dikelompokkan, program menghitung jumlah kuadrat dari dua sisi terpendek (sisi1 dan sisi2) dan menyimpannya dalam variabel kuadratHipotenusa. Program kemudian membandingkan apakah nilai tersebut sama dengan kuadrat dari max_val. Jika nilainya sama, fungsi mengembalikan nilai true, yang berarti ketiga bilangan tersebut memenuhi syarat sebagai Tripel Pythagoras.

```
int main() {
    int a, b, c;

    cout << "Masukkan bilangan a: ";
    cin >> a;
    cout << "Masukkan bilangan b: ";
    cin >> b;
    cout << "Masukkan bilangan c: ";
    cin >> c;

    if (cekTripelPythagoras(a, b, c)) {
        cout << "Tripel Pythagoras." << endl;
    } else {
        cout << "Bukan Tripel Pythagoras." << endl;
    }

    return 0;
}
```

Gambar 3.4.3 Source Code main(), input bilangan a, b, dan c

Pada fungsi utama, program meminta input tiga bilangan bulat dari pengguna. Program kemudian memanggil fungsi cekTripelPythagoras di dalam sebuah struktur kontrol if. Jika fungsi tersebut menghasilkan nilai true, program akan menampilkan teks "Tripel Pythagoras". Sebaliknya, jika hasilnya false, maka program menampilkan "Bukan Tripel Pythagoras".

5. Output

```
Masukkan bilangan a: 3
Masukkan bilangan b: 4
Masukkan bilangan c: 5
Nilai kuadrat dari sisi miring (hipotenusa) = 25
Tripel Pythagoras.
```

Gambar 3.5 Output

Deskripsi Tugas :

1) Buatlah fungsi pythagoras.
2) Terdapat 3 inputan yang berupa bilangan bulat a, b, c.
3) Tentukan nilai pytagorasnya.
4) Tentukan apakah ketiga bilangan tersebut merupakan tripel pythagoras atau tidak.
5) Buatlah pseudocode, algoritma dan flowchart.

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

BAB IV

1. Pseudocode

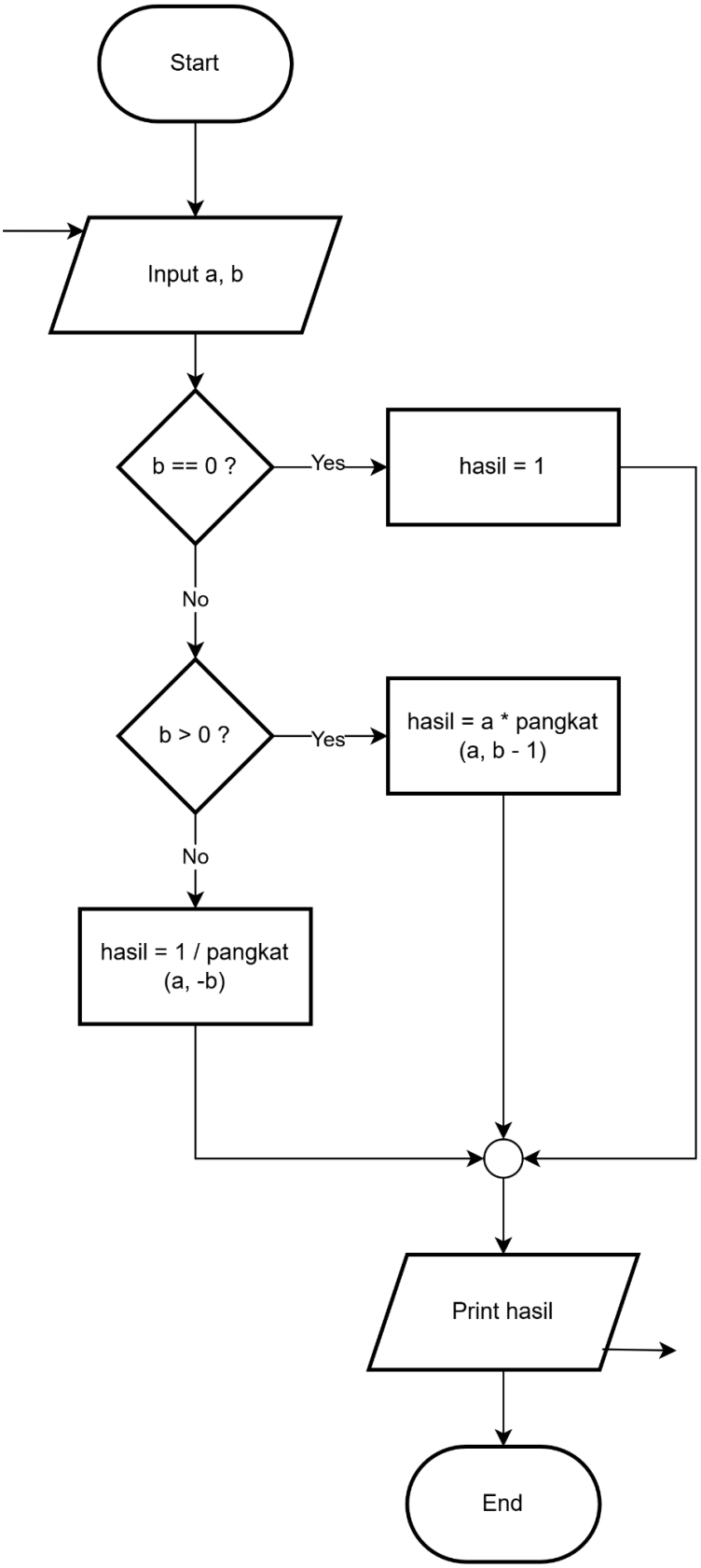
```
Judul :
Program Fungsi Rekursif untuk Menghitung Pangkat Bilangan

Kamus :
a, b : integer

Algoritma :
start
   input a, b
   if b == 0 then
      hasil = 1
   else if b > 0 then
      hasil = a * pangkat(a, b - 1)
   else
      hasil = 1 / pangkat(a, -b)
   end if
   print "Hasil dari " + a + "^" + b + " adalah: " + hasil
end
```

Gambar 4.1 Pseudocode

2. Algoritma

1) Meminta pengguna memasukkan dua bilangan bulat: basis (a) dan eksponen b).
2) Pemanggilan Fungsi Rekursif Melemparkan nilai a dan b ke dalam fungsi pangkat.
3) Proses Rekursi
    a) Periksa apakah eksponen adalah nol. Jika ya, berikan hasil 1.
    b) Jika eksponen positif, lakukan perkalian berulang dengan memanggil fungsi itu sendiri sambil mengurangi nilai eksponen secara bertahap (b-1).
    c) Jika eksponen negatif, ubah kalkulasi menjadi pembagi (1 ÷ hasil pangkat positif).
4) Setelah tumpukan rekursi selesai (*unwinding*), fungsi mengirimkan hasil akhir ke fungsi utama.
5) Menampilkan hasil perhitungan perpangkatan ke layar.

3. Flowchart



Gambar 4.3 Flowchart

4. Sourcecode

```
#include <iostream>
using namespace std;

double pangkat(int a, int b) {
    if (b == 0) {
        return 1;
    } else if (b > 0) {
        return a * pangkat(a, b - 1);
    } else {
        return 1.0 / pangkat(a, -b);
    }
}
```

Gambar 4.4.1 Source Code fungsi rekursif

Program menggunakan fungsi pangkat(a, b) dengan pendekatan Rekursif, yaitu fungsi yang memanggil dirinya sendiri. Terdapat tiga kondisi utama:

1) Jika pangkat (b) adalah 0, maka fungsi mengembalikan nilai 1 (karena bilangan apapun pangkat 0 hasilnya 1).
2) Recursive Step (Positif): Jika b > 0, fungsi akan mengalikan a dengan hasil dari pangkat(a, b-1). Proses ini akan terus berulang hingga b mencapai 0.
3) Handling Pangkat Negatif Jika b < 0, fungsi menggunakan logika $a^{-b} = \frac{1}{a^b}$ dengan memanggil kembali fungsi pangkat menggunakan nilai b yang di-positifkan, sehingga program tetap akurat untuk pangkat negatif.

```
int main() {
    int a, b;

    cout << "Masukkan bilangan (a): ";
    cin >> a;
    cout << "Masukkan pangkat (b): ";
    cin >> b;

    cout << "Hasil dari " << a << "^" << b << " adalah: " << pangkat(a, b) << endl;

    return 0;
}
```

Gambar 4.4.2 Source Code main(), input bilangan dan pangkat serta output

Dalam fungsi utama, program mendeklarasikan variabel a sebagai basis dan b sebagai eksponen. Program meminta input dari pengguna melalui cin, lalu memanggil fungsi pangkat(a, b). Karena fungsi pangkat bertipe double, hasil perpangkatan dapat ditampilkan dengan benar meskipun hasilnya berupa bilangan pecahan (pada kasus pangkat negatif).

5. **Output**

```
Masukkan bilangan (a): 5
Masukkan pangkat (b): 3
Hasil dari 5^3 adalah: 125.
```

Gambar 4.5 Output

Deskripsi Tugas :

1) Buatlah fungsi untuk mencari hasil pangkat dari suatu bilangan dengan menggunakan rekursif.
2) Terdapat dua inputan berupa bilangan bulat a dan b.
3) a untuk inputan bilangan, b untuk inputan pangkat.
4) Tentukan hasil pangkatnya.

Nim dan Nama yang mengerjakan :

2509106006 - Muhammad Alfauzi Syahputra

BAB V
KESIMPULAN

Berdasarkan pelaksanaan Tugas 1 dan penyusunan laporan mengenai algoritma pemrograman lanjut ini, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Pemanfaatan Array 1 Dimensi dan Multidimensi**

Penggunaan array sangat efektif dalam mengelola sekumpulan data sejenis. Array 1 dimensi memungkinkan pengolahan deret angka seperti bilangan prima dan Fibonacci secara sistematis. Sementara itu, array multidimensi (matriks) memungkinkan representasi data yang lebih kompleks, seperti koordinat ruang atau tabel data, serta mempermudah operasi matematika seperti perhitungan invers matriks, pencarian nilai ekstrem (min/max), dan perhitungan rata-rata.

2. **Logika Struktur Kontrol pada Matriks**

Pembentukan pola visual seperti matriks segitiga memerlukan pemahaman yang kuat mengenai hubungan antara indeks baris (i) dan indeks kolom (j). Penggunaan diagonal sekunder sebagai acuan kondisi (i + j) membuktikan bahwa logika matematika dasar sangat krusial dalam memanipulasi tata letak data pada array dua dimensi.

3. **Efisiensi dengan Pointer dan Struktur (Struct)**

Penggunaan struct memungkinkan pengelompokan berbagai tipe data menjadi satu entitas (objek), seperti pada data Player. Implementasi Pointer dalam mengakses data tersebut memberikan efisiensi memori yang lebih baik karena program hanya mengirimkan alamat memori (reference) ke dalam fungsi/prosedur, bukan menduplikasi seluruh isi data.

4. **Modularitas melalui Prosedur dan Fungsi**

Pembagian program ke dalam prosedur (seperti penentuan jenis segitiga) dan fungsi (seperti cek Tripel Pythagoras) membuat kode menjadi lebih modular, mudah dibaca, dan dapat digunakan kembali (reusable). Hal ini memisahkan antara logika perhitungan dengan alur utama program (main).

5. **Kekuatan Rekursi**

Fungsi rekursif merupakan solusi yang elegan untuk permasalahan yang memiliki pola berulang, seperti perpangkatan. Dengan menentukan base case (kondisi berhenti) dan recursive step yang tepat, permasalahan kompleks dapat dipecah menjadi sub-masalah yang lebih sederhana. Penanganan pangkat negatif dalam fungsi rekursif juga menunjukkan fleksibilitas algoritma dalam menangani berbagai variasi input.

DAFTAR PUSTAKA

Belajar Pointer C++: Dasar-Dasar, Fungsi, dan Contoh Kode - CODEPOLITAN. (2024, December 10). Codepolitan. Retrieved February 16, 2026, from https://www.codepolitan.com/blog/belajar-pointer-c-dasardasar-fungsi-dan-contoh-kode/